Knowledge Distillation and Kubernetes Deployment of a Command-Parsing Model

CS4032 Cloud Computing
Assessment 3

Submitted By

Yogita Kumari
SEMESTER - VII
230101091

Supervised By

Dr. Salam Michael Singh



Department of Computer Science and Engineering
Indian Institute of Information Technology (IIIT) Senapati, Manipur
6 November, 2026

GitHub repository: `https://github.com/yogitaranwa/CS4032A3`

1 Architecture

The project turns a pool of 3,000 short voice-assistant commands into a small model that returns a JSON frame with four keys: intent, date, time and place. A teacher model, phi3.5 served by Ollama, labels the commands. The labels are cleaned and used to fine-tune a student, EleutherAI pythia-70m, on CPU. The student runs behind a FastAPI service on a single-node k3d Kubernetes cluster. SeaweedFS provides S3-compatible object storage in the namespace `data`. In the 30 October run the 3,000 commands were labelled by 15 GitHub Actions jobs, each running Ollama on its own runner; the same worker code also runs as Kubernetes Jobs in the namespace `pipeline`, which was demonstrated on 45 commands. The service, its Service, a Traefik ingress and a CPU-based Horizontal Pod Autoscaler (HPA) run in the namespace `app`. Model files live in the bucket `models`. The released models are also GitHub Release assets with SHA-256 checksums, so the repository holds no model files. Figure 1 in Appendix B shows the layout as a schematic.

The heavy stages run as GitHub Actions workflows (`label-teacher`, `train`, `deploy-and-measure`, `ci`), and each commits its evidence to `results/`.

Data and control flow. The flow has six steps. First, `commands_pool.jsonl` is split by line number into 15 shards (the cluster demo also uploads it to the `datasets` bucket). Second, the sharded workers read their lines, call the teacher and write one JSONL file per shard. Third, `clean.py` merges the shard files, applies the rejection rules and writes `train.jsonl` with its checksum. Fourth, the training job fine-tunes the student and packs each model version into an archive. Fifth, the archive is published as a Release asset and copied to the `models` bucket, from where the service downloads it at start. Sixth, the Deployment, Service, ingress and HPA serve and scale the model, and the evidence scripts drive the load tests against the ingress. Each step reads only the output of the step before it, so any step can be repeated without rerunning the others.

Component	Where it runs	What it stores or does
Ollama with phi3.5	the runner (full run) or namespace `pipeline` (cluster demo)	Teacher; temperature 0, seed 42
SeaweedFS	namespace `data`	Buckets `datasets` and `models`
Labelling workers	15 runner jobs (full run) or Jobs in `pipeline` (demo)	One shard file: 200 commands per runner job, 3 per demo Job
Trainer	Runner or laptop, CPU only	Student model, loss file, archive and checksums
FastAPI service	Deployment in `app`, 1 to 2 replicas	`/parse`, `/healthz`, `/readyz`
Traefik ingress	Namespace `app`	Single entry point and load balancing
HPA	Namespace `app`	70% CPU target, 1 to 2 replicas

Provenance of the measurements. Every measured value in this report was copied from a file in `results/`. Those files are the raw output of the scripts in this repository (kubectl, Locust and scoring output, with the runner host name and UTC timestamps of 30 October 2026). They come from one complete earlier run of this same code on a GitHub-hosted runner in a separate rehearsal repository. They have not yet been regenerated in this repository, `yogitaranwa/CS4032A3`, so no number below was produced by a run in this repository. The validation scores and the cleaning counts were re-derived from the stored prediction and shard files with `tools/score.py` and `worker/clean.py`, and they match the stored results. Statements marked as assumptions were not measured. The runner has two vCPUs shared by the cluster, the load generator and the replicas, so absolute latencies are higher than on a dedicated machine.

2 Task 1: Object Storage and Parallel Teacher Labelling

Architecture. Storage and labelling form one pipeline on the k3d cluster: SeaweedFS holds the datasets and models buckets, and the `pipeline` namespace runs 15 labelling workers that each read

one shard and call the Ollama teacher.

Implementation. The k3d cluster is defined in `cluster/k3d.yaml`. SeaweedFS runs in `data` with the two buckets, and credentials are generated by `scripts/secrets.sh` into Kubernetes Secrets, never into the repository. The sharding rule is the line number of a command modulo `SHARD_COUNT`. With `SHARD_COUNT` of 15 each shard holds 200 commands and runs as one job on one runner, which keeps the shards equal in size and lets the 15 jobs run in parallel. Each worker sends the prompt in `prompts/teacher_prompt.txt` to phi3.5 at temperature 0 with seed 42 and stores the reply exactly as generated. `worker/clean.py` first checks that every id appears in exactly one shard, then applies four rejection rules in order (invalid JSON, wrong keys, intent outside the eight allowed values, slot text that is not a phrase of the command), and finally removes duplicates, keeping the smallest id. The in-cluster run with 15 workers in `results/pipeline/` is a short demonstration (3 commands per shard, 45 in all), not the full labelling. The 15 shard files of 200 commands each are in `datasets/teacher/`.

Prompt design. The teacher prompt lists the eight allowed intents with one example command each (alarm_set, alarm_query, calendar_set, calendar_query, calendar_remove, weather_query, datetime_query and recommendation_events). It tells the teacher to choose the single closest intent even when the command is vague, and to reply with one JSON object with exactly the keys intent, date, time and place in that order. The slot rules say that a value is a phrase copied word for word from the command, in lower case, without words such as at, on or in, and that a missing slot is JSON null. Numbers and dates must not be converted, so "seven am" stays "seven am". These rules exist because the cleaning rule (d) rejects any slot that is not text found in the command. The few examples in the prompt were written by hand and do not come from the validation file.

Results (measured, `results/data_counts.json`).

Item	Count
Input records	3000
Rejected: invalid JSON / wrong keys / bad intent	0 / 0 / 0
Rejected: slot not in the command	152
Duplicates removed	44
Kept	2804

The counts add up: $0 + 0 + 0 + 152 + 44 + 2804 = 3000$. The SHA-256 of `datasets/v1/train.jsonl` is `8646e60b...c776c1a3`, stored in `train.sha256`. The cleaned set has a skewed intent mix, which matters for Task 2. The table below counts the intents in `train.jsonl` (measured, 2804 records) and shows how many records carry each slot.

Intent	Records	Slot	Records with a value
calendar_set	735	date	981
weather_query	506	time	453
calendar_query	453	place	381
recommendation_events	353		
calendar_remove	310		
alarm_set	186		
datetime_query	176		
alarm_query	85		

The training set is dominated by calendar_set (26%), and alarm_query has only 85 records (3%). The validation set, in contrast, has 12 or 13 commands of each intent.

Limitations and improvements. The 152 slot rejections show that the teacher sometimes paraphrases a slot instead of copying words, so about 5% of the pool is lost. A second pass that asks the teacher to repair only the rejected records, or constrained decoding to a JSON grammar, would recover many of them. Whether the recovered records are as clean as the kept ones was not measured.

3 Task 2: Training the Student

Architecture. Training is one job that reads the cleaned training file from the bucket, fine-tunes the student on it and writes the weights and tokenizer files back to the models bucket.

Implementation. `training/config.yaml` is the only source of settings. Training uses the first 1,500 cleaned records, seed 42, learning rate 10^{-4}, batch size 16, maximum sequence length 128, 5 epochs for v1 and 6 for v2, with torch 2.4.1 and transformers 4.44.2 pinned. The input is "Command: ⟨text⟩\nJSON:" and the loss is computed on the reply tokens only. The model archive is built reproducibly. Its SHA-256 for v1 is `7c6259e5...a55a2` (`models/v1/model.sha256`), and the service verifies `files.sha256` after download.

Results (measured, 100 validation commands, `results/*_validation.json`).

Model	Valid JSON	Intent	Date	Time	Place	Full frame
Teacher phi3.5	100	76	79	86	95	48
Pythia-70M untrained	0	0	0	0	0	0
Student v1 (5 epochs)	99	73	82	88	92	50
Student v2 (6 epochs)	99	77	80	87	91	52

The training loss falls from 0.358 to 0.018 over five epochs for v1 (`training/loss.csv`) and from 0.364 to 0.012 over six epochs for v2 (`training/loss_v2.csv`).

Sequence-level versus logit-based distillation. Sequence-level distillation trains the student on the text the teacher generated, as ordinary supervised fine-tuning. Logit-based distillation trains the student to match the teacher's probability distribution over the next token, which carries more information per example. It is not possible here because the teacher is served through Ollama as text, and the teacher and student use different tokenizers and vocabularies, so their output distributions have no common index. The cost of the sequence-level choice is that the student inherits the teacher's mistakes as hard labels, with no signal about how confident the teacher was.

Limitations and improvements. The student matches the teacher on full frames (50 and 52 against 48), but the difference is two or four commands out of 100 and is within the noise of such a small set. The difference between v1 and v2 is also within that noise: v2 is better on intent and full frame and worse on date, time and place, so the data do not show that the sixth epoch helps. A larger validation set and several seeds would be needed to say more. Intent accuracy of 73 to 77, close to the teacher's 76, suggests that the quality of the teacher labels limits the result more than the student size does; this was not tested separately.

Error analysis (measured, from the stored predictions and the gold labels in `data/validation.jsonl`). The most frequent intent error of the teacher and both students is to answer calendar_set for a command that asks about the calendar or the time. Student v1 gives calendar_set instead of calendar_query on 7 commands, student v2 on 6, and the teacher on 6. All three give calendar_set instead of datetime_query on 4 or 5 commands. The students also confuse alarm_query with alarm_set (4 commands for v1, 2 for v2, 3 for the teacher). These confusions follow the training mix, because calendar_set and alarm_set are the larger classes. On the slots, most errors are a missing or an extra value, not different wording. For student v1 there are 12 date, 8 time and 6 place errors of this kind against 5, 3 and 1 errors where both sides have a value but the text differs. Student v2 has 11, 9 and 7 such errors, against 8, 3 and 1. So the student rarely changes the words of a slot it extracts, which shows that the copy rule of the prompt was learned, and the main weakness is deciding whether a slot exists.

Why the loss does not predict the score. The loss of v2 is lower than that of v1 (0.012 against 0.018), but the full-frame score changes by only two commands. The loss is measured on the training reply tokens, where the model sees the teacher's own words, and the validation labels are the gold labels of the course dataset, which the teacher itself scores only 48 on. A lower training loss therefore mostly measures how closely the student copies the teacher, so it is not evidence of agreement with the gold labels.

4 Task 3: The Inference Service

Architecture. The service is a stateless FastAPI container that downloads the model from the models bucket at start, checks the checksums and then answers `/parse` from memory.

Implementation. The FastAPI service exposes `GET /healthz`, `GET /readyz` and `POST /parse`. The model loads in a background thread, and `/readyz` returns 503 until it is ready, so Kubernetes sends no traffic to a Pod that is still loading. Every reply carries the headers `X-Model-Version` and `X-Served-By`. The generated text is validated against the schema, and an invalid output returns HTTP 500 with `invalid_model_output` instead of passing bad JSON on. Generation is limited to 64 new tokens. The setting `FAULT_INJECT=1` replaces the output with invalid text to test this path. On start the service downloads the model from the `models` bucket using credentials from a Secret and verifies the checksums.

Results (measured, `results/memtest/`). In a container limited to 1 CPU and 1 GiB, 50 of 50 requests returned HTTP 200, the container was not OOM-killed, and peak memory was 358.7 MiB. The image is 1.22 GB (1,215,604,063 bytes), under the 2 GB limit. The startup log shows the download of `model.safetensors` (281,715,176 bytes) and the tokenizer files from the bucket.

Behaviour of the first 50 requests (measured, `results/memtest/requests.log`). The 50 requests are the first 50 commands of the validation file (`scripts/memtest.sh` reads them in order). All 50 returned HTTP 200 with a body that passed the schema check, and none returned the `invalid_model_output` error. Request 4, for example, asked "what time does mondays meeting start" and received calendar_set with no slots, which is a wrong answer that is still valid JSON. This shows the limit of the validation gate: it checks the shape of the reply and not its truth, so the correctness of the service is measured only by the scoring script in Task 2.

Configuration and security. The service reads its model version, model location, object-storage address and generation limit from environment variables, and its credentials from a Kubernetes Secret that `scripts/secrets.sh` generates at deploy time with `openssl`. The `.secrets` folder is excluded by `.gitignore` and `.dockerignore`, so the credentials are in neither the repository nor the image. The container checks every downloaded file against `files.sha256` before it loads the model, so a damaged or replaced file raises an error in the loading thread and the service never becomes ready, instead of serving a wrong model.

Limitations and improvements. The image contains the PyTorch CPU runtime, which is assumed (not measured) to account for most of its size. An ONNX or quantised export would shrink the image and cut latency. Generation is sequential per request, and no batching is implemented.

5 Task 4: Kubernetes Deployment, Scaling and Reliability

Architecture. Traefik routes requests from the ingress to the Service in front of the Deployment, whose Pods are replaced by rolling updates and added or removed by the horizontal Pod autoscaler.

Implementation. The Deployment starts with 2 replicas, requests 500m CPU and 512 Mi memory and limits 1 CPU and 1 Gi. The readiness probe calls `/readyz` every 3 s, and the liveness probe calls `/healthz` after 15 s. The HPA targets 70% CPU with a minimum of 1 and a maximum of 2 replicas, and its scale-down window is 60 s. Traefik exposes the Service through an ingress.

Results (measured; Locust with 4 users, files in `results/`).

Test	Result	Evidence
Readiness	The endpoint list stayed empty while the new Pods loaded the model, and each Pod was added only after it became ready (1/1)	`k8s/readiness.txt`
Autoscaling	CPU rose to 185% of the request within about 10 s of the load start, the HPA went from 1 to 2 replicas, and it returned to 1 about 80 s after the load ended; 0 of 615 requests failed	`hpa/`
Rolling update v1 to v2	Completed; 1 of 906 requests failed (0.110%); responses showed both versions during the update	`rollout/`
Rollback	After the rollback the ingress returned v1 for all but the first sampled reply	`rollout/commands.txt`
Pod deletion	A deleted Pod was replaced and ready in 7 s; 0 of 708 requests failed	`recovery/`
Cold start (3 runs)	Slowest Pod per run 18, 17 and 17 s (median 17 s); median over all Pods 12 s	`coldstart/`
Load balancing	248 and 250 requests to the two replicas	`standard/`

Latency in each scenario (measured, `loadtest_stats.csv` of each folder). Each scenario used the same Locust settings of 4 users, a spawn rate of 4 and commands taken in turn from the validation file. A request counts as failed when it does not return HTTP 200 within 30 s. Figure 2 in Appendix B plots these values.

Scenario	Requests	Failures	Median (ms)	p95 (ms)	p99 (ms)
Standard (2 min)	496	0	950	1500	1600
Autoscaling	615	0	1100	1700	2100
Rolling update	906	1	1100	1800	2100
Pod deletion	708	0	960	1600	2000

Compared with the standard run, the median is 10 to 150 ms higher in the disturbed scenarios and the p95 is 100 to 300 ms higher. During the autoscaling run the two replicas used about 930 to 990m CPU each, so each replica was close to its limit of one CPU, and the runner had no spare capacity for the load generator. This is a likely cause of the higher median in that run, but the run did not isolate it.

Design of the scenarios. The readiness test deletes all Pods and samples the Pod list and the Service endpoints every second. The rolling update changes the model version through environment variables, waits for the rollout, and then asks the ingress six times which version answers. The rollback uses `kubectl rollout undo` and repeats the check eight times. The recovery test deletes one Pod and records the time until two replicas are ready again. The cold-start test deletes all Pods three times and extracts the download and load times from the logs of each new Pod.

Limitations and improvements. The cluster has one node, so Pod failure is shown but machine failure is not. The rolling update lost one request. The shipped manifest sets `maxUnavailable` to 0 and a 10 s `preStop` delay, but the stored results do not show whether the 30 October run used them, so the cause is not established. One failed request in 906 is a single event and does not measure the true failure rate. The maximum of 2 replicas is set by the runner's two vCPUs, so the autoscaling behaviour at larger scale is an assumption, not a measurement.

6 Task 5: Subnetting and Network ACL

Architecture. The address block is split into four subnets (app, public, data and management), and a stateless Network ACL guards the public subnet.

The block is 10.20.0.0/16. Subnets are allocated largest first, each with the smallest prefix whose usable host count $(2^h - 2)$ covers the need, and each placed directly after the previous one.

Subnet	Hosts	Prefix	Network	Usable range	Broadcast
app	1000	/22	10.20.0.0	.0.1 – .3.254	10.20.3.255
public	250	/24	10.20.4.0	.4.1 – .4.254	10.20.4.255
data	100	/25	10.20.5.0	.5.1 – .5.126	10.20.5.127
management	20	/27	10.20.5.128	.5.129 – .5.158	10.20.5.159

A /23 would give only 510 hosts for the app subnet, so /22 (1,022 hosts) is the smallest fit; likewise /25 gives 126 hosts for data and /27 gives 30 for management. Every network address is a multiple of its block size.

The Network ACL of the public subnet is in `network/nacl.json`. Inbound rule 100 denies 198.51.100.0/24, rules 110 and 120 allow TCP 80 and 443 from anywhere, rule 130 allows SSH from the management subnet only, and rule 140 allows TCP 1024–65535 from the app subnet for replies. Outbound rule 100 allows TCP 8000 to the app subnet and rule 110 allows TCP 1024–65535 to anywhere for replies to clients. Anything else hits the implicit deny. A network ACL is stateless, so a reply is judged as a separate packet and needs its own rule for the ephemeral port range. Rules are evaluated from the lowest number and the first match wins, so the deny must have a lower number than the allows; if it were numbered above 110, traffic from the abusive range to port 443 would be allowed. `tests/test_network.py` recomputes the subnet table with Python's `ipaddress` module and `tools/nacl_sim.py -check` runs the rules against `network/packets_sample.jsonl`. The check is part of the CI workflow. This task is a design exercise, so no result in it was measured on a real cloud network. The /22 for the app subnet leaves room for growth; the 16-bit block still has most of its space unallocated.

Simulator result (measured, `tools/nacl_sim.py -check`). The six sample packets give the expected decision in all six cases (6 of 6).

Packet	Direction	Port	Address	Decision	Rule
S01	inbound	443	192.0.2.55	allow	120
S02	inbound	22	192.0.2.55	deny	default
S03	outbound	8000	10.20.1.1	allow	100
S04	inbound	443	198.51.100.9	deny	100
S05	outbound	60000	192.0.2.55	allow	110
S06	inbound	5432	10.20.5.20	deny	default

Packet S04 shows the rule order: the packet is to port 443, which rule 120 would allow, but rule 100 matches first and denies it. Packet S02 shows that SSH from outside the management subnet is denied by the default. Packet S06 shows that a database port is not open to the data subnet through this ACL. The sample is small and was supplied with the kit, so it tests the intended behaviour and does not search for gaps. One limit of the design is that a network ACL cannot name a service, so the rules allow whole port ranges for replies. A security group, which is stateful, would remove the ephemeral-port rules for the load balancer.

7 Task 6: Reproducibility, Testing and CI/CD

Architecture. Automation has two layers: a Makefile for local commands and GitHub Actions workflows that run the heavy stages and commit their evidence to `results/`.

Implementation. A fresh clone is deployed with three commands: clone, `make up` and a test request; `make down` deletes the cluster. `make up` builds the images, starts SeaweedFS, creates the buckets, loads the models and applies the manifests. The repository has 36 unit-test functions in seven files covering sharding and the rejection rules, the reproducible checksum, the service endpoints and fault injection, the manifests, the scoring script and the network answers. The `ci` workflow runs ruff, a parse check of all JSON and YAML files, the NACL check, the unit tests, a model evaluation from the Release asset, a quality gate and a gitleaks secret scan. The gate requires a JSON validity of at least 95%. None of the reported measurements fails this gate except the untrained base model, which is the intended negative case.

Results (measured with the scoring script). The untrained base model has a validity of 0% (`results/base_validation.json`) and therefore fails the 95% gate, and student v1 and v2 score 99% and pass it (table in Task 2). A `ci` run with `model = untrained-base` is the intended failed-gate demonstration; this report does not cite such a run, so the scores above are the measured evidence for the gate.

Build, deployment, testing and shutdown. The commands below are documented in `README.md` and defined in the `Makefile`.

Goal	Command and effect
Build and deploy	`make up`: creates the k3d cluster, builds the images, starts SeaweedFS and its buckets, downloads or loads the models, and applies the manifests
Test	`make test` runs the unit tests, `make lint` runs ruff, `make nacl-check` runs the ACL check
Train	`make train` for v1, `make train VERSION=v2` for v2, `make train SUBSET=50` as a smoke test
Label in the cluster	`make label-cluster` runs the workers in the namespace `pipeline`
Measure	`make evidence` runs the Task 4 and Task 7 scenarios on a running cluster and writes `results/`
Shut down	`make down` deletes the cluster and all its data

The workflows split the same work into stages. The `label-teacher` workflow runs 15 shard jobs, scores the teacher, merges and cleans the shards and runs a cluster demo. The `train` workflow trains v1 and v2, evaluates the untrained base model and publishes the Release. The `deploy-and-measure` workflow runs the memory test, `make up`, the Task 4 scenarios and the standard load test. The `validate` workflow is run on demand. Each workflow commits its evidence files back to `main`. Jobs hold only the permissions they need, which is read access for `ci` and write access for the workflows that commit results. A workflow that commits evidence while the previous one is still pushing can conflict, so the workflows are run one after another.

Limitations and improvements. The gate checks only JSON validity. Adding a floor for full-frame accuracy would catch a model that produces valid but wrong frames. The gate was applied to a 100-example validation set, which is small.

8 Task 7: Performance, Cost and Review

Architecture. Locust drives four closed-loop users at the ingress while `kubectl` and `top` sample the Pods.

Measurements (standard Locust run, 4 users for 2 minutes, `results/standard/`). The standard run completed 496 requests with no failures, a median of 950 ms, a 95th percentile of 1500 ms and a 99th percentile of 1600 ms, which is 4.16 requests per second. A Pod used 2m of CPU when idle and 920 to 990m under load (`top.txt`, `hpa/timeline.txt`). Cold start was 17 s.

Cost (calculated from the hypothetical rate card in `data/rate_card.json`, an assumption). A replica is billed for its limits of 1 vCPU and 1 GiB, which costs $3.50+0.45 = 3.95$ INR per hour. At the measured 4.16 requests per second, which is 14,984 requests per hour, the cost is $7.90/14984 \times 1000 \approx 0.53$ INR per 1,000 requests. This holds only while both replicas are busy, because an idle replica is billed in the same way.

Service level objective (a proposal and an assumption). A reasonable objective for this service on this hardware is that 95% of requests complete within 2 s and that fewer than 0.5% of requests fail. The measured values meet it in all four scenarios (p95 of 1.5 to 1.8 s, failure rate 0% to 0.110%). Limits: small validation set, one node, shared runner, hypothetical rates. Improvements: a saturation test, a multi-node cluster and real prices.

References

Bastianelli, E., Vanzo, A., Swietojanski, P., Rieser, V. SLURP: A Spoken Language Understanding Resource Package. EMNLP 2020.
Kim, Y., Rush, A. M. Sequence-Level Knowledge Distillation. EMNLP 2016.

Appendix A: Evidence files in `results/`

Path	Content
`data_counts.json`	Cleaning counts (Task 1)
`teacher_validation.json`, `base_validation.json`, `student_v1_validation.json`, `student_v2_validation.json`	Scores of the four models (Tasks 2 and 6); the matching `*_predictions.jsonl` files hold the 100 predictions of each
`memtest/`	50-request test at 1 CPU and 1 GiB, image size, peak memory, start-up log (Task 3)
`k8s/`	Resources, probes, node list and readiness samples (Task 4)
`hpa/`, `rollout/`, `recovery/`, `coldstart/`	Timelines, Locust output and command output of the Task 4 scenarios
`standard/`	Standard Locust run, `top` samples and header counts (Task 7)
`pipeline/`	Short cluster demo of the labelling workers, with Ollama and SeaweedFS logs (Task 1)

Appendix B: Figures

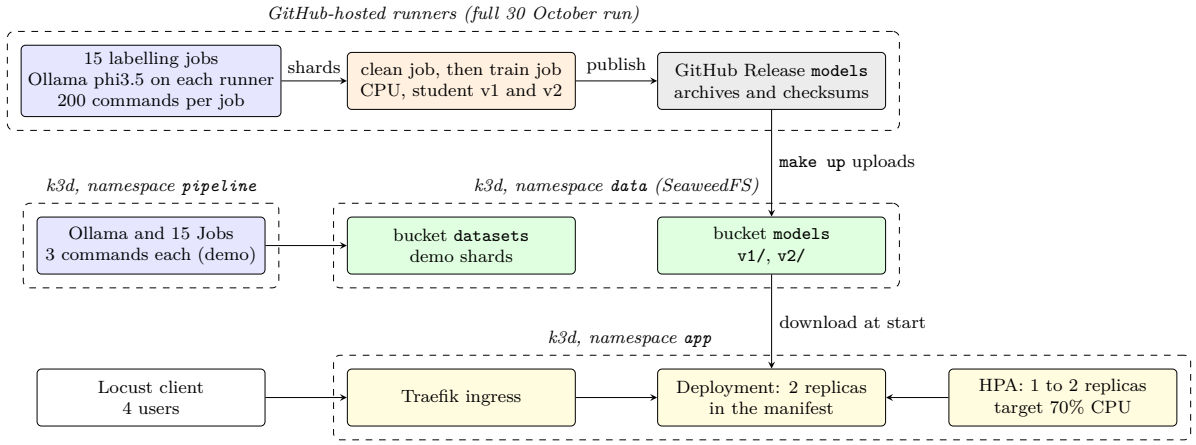


Figure 1. Schematic of the system. The top row is the full labelling and training on GitHub-hosted runners, with the model published as a Release asset. The lower rows are the k3d cluster: the in-cluster labelling demo wrote 45 commands to the `datasets` bucket, and `make up` copies the released model into the `models` bucket, from which the Pods download it.

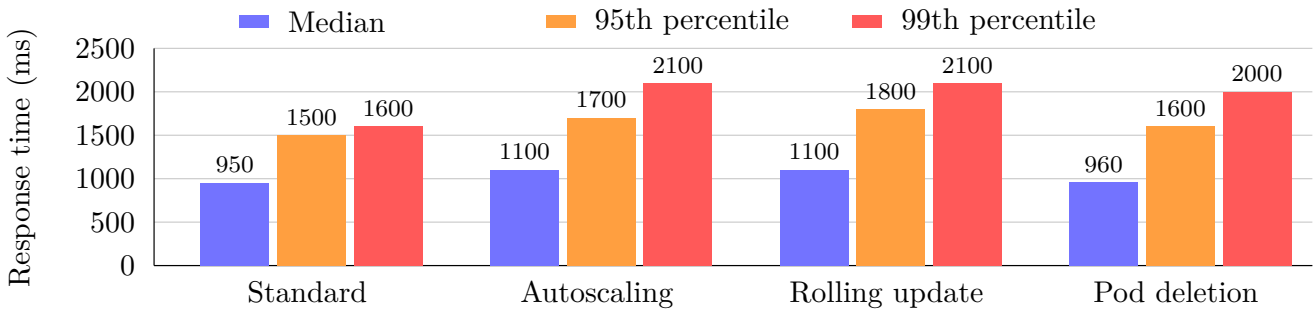


Figure 2. Measured response times of the four Locust scenarios (4 users, results of 30 October 2026, files `loadtest_stats.csv` in `results/standard`, `hpa`, `rollout` and `recovery`). Requests per scenario: 496, 615, 906 and 708; failures: 0, 0, 1 and 0. The values are the same as in the table of Task 4.